Permutations Are Not Enough: Sign Flips Explain the Linear-Mode-Connectivity Barrier in Minimal-Width Sigmoid Networks

Mrigesh Lakhanpal
mrigeshlakhanpal4@gmail.com
ORCID: 0009-0007-9539-9091

October 2026 (preprint, v1)

This work is licensed under a Creative Commons Attribution 4.0 International license (CC BY 4.0).

Abstract

Why permutation alignment fails to linearly connect narrow networks remains, to my knowledge, an open question. Ainsworth et al. raised the possibility that either their matching methods fall short on thin models or "some form of invariance other than permutation symmetries" is at play, and Benzing et al. noted that settling the narrow case would require an exhaustive search, which they considered infeasible. One-hidden-layer sigmoid networks make that search possible, because for irreducible networks the complete group of exact symmetries is known and finite: permutations of hidden units combined with sign flips, $2^h h!$ elements at width h. I trained 40 such networks per width (2 to 64) on a two-dimensional classification task and measured the training-error barrier along the straight line between independently trained pairs. At width 3, the smallest width that solved the task, all 11 pairs learned the same three decision lines, yet permutation matching left a median barrier of 17.95 points of training error; adding sign flips removed it (median 0.00). A follow-up exhaustive search over all 48 symmetries, chosen by the full barrier, left at most 0.11 points in every pair, and all networks involved were irreducible, so the search was complete. For these networks, the evidence therefore points to "another symmetry": the sign flip. A short proof (Proposition 1) shows that a single spare neuron joins a network to all of its $2^r r!$ symmetry copies by a path along which the function is exactly constant. The study is deliberately small and limited to one architecture and one task, but its main questions were fixed before training, and one command rebuilds every result except the multi-hour exhaustive search, reproducing the main result files byte for byte.

Keywords: linear mode connectivity; permutation symmetry; sign-flip symmetry; sigmoid networks; loss landscape; model merging

1 Introduction

Two networks trained from different random initializations usually reach similar accuracy, yet the straight line between their weights typically passes through a region of high loss. Entezari et al. [1] conjectured that most of this barrier is an artifact of permutation symmetry: reorder the hidden units of one network appropriately, and SGD solutions become linearly mode connected [2]. Ainsworth et al. [3] gave strong evidence for wide networks, reaching zero-barrier connections between independently trained ResNets once they were wide enough. In narrow networks, however, permutation alignment does not remove the barrier, and, as far as I am aware, the reason has not yet been settled. Ainsworth et al. write that their result "suggests that either our permutation selection methods are failing to find satisfactory permutations on thinner models or that

some form of invariance other than permutation symmetries must be at play in the thin model regime" [3, Section 5.3], and they conjecture that permutation symmetries are "a necessary piece, though not a complete picture" [3, Section 7]. Benzing et al. observe the same failure in narrow fully connected networks and add that "a thorough falsification of the hypothesis in narrower networks would require an exhaustive (and therefore infeasible) search over the space of permutations" [4, Section 3.1]. Entezari et al. note that small networks lie outside the scope of their conjecture [1, Section 3.4].

In general this question is difficult to settle, because the complete symmetry group of a network is either unknown or too large to search. One-hidden-layer sigmoid networks are an exception. For irreducible networks of this kind, the only exact symmetries are permutations of the hidden units and sign flips of individual units, which follow from $\sigma(-z) = 1 - \sigma(z)$ [5, 6]. The group has $2^h h!$ elements, only 48 at width $h = 3$, so "matching failed" and "another symmetry is involved" can be told apart by trying every element.

I used this to study independently trained networks on a two-dimensional task whose decision region is a triangle, so that the smallest working network has three hidden units and every learned feature is a line that can be drawn. My contributions are:

1. **Sign flips, not matching failure, explain the barrier at minimal width** (pre-registered; Section 4.2). At width 3, permutation matching leaves a median training-error barrier of 17.95 points, while matching with sign flips removes it (median 0.00). The best alignment with flips lowered the barrier by more than 0.5 points in 10 of 11 pairs at width 3, and in 16 of 20 at widths 4 and 6.
2. **An exhaustive search over the complete exact symmetry group removes the barrier** (post-hoc, rules dated before the search; Section 4.3). Searching all 48 permutation-and-flip combinations at width 3 and choosing by the full barrier leaves at most 0.11 points in all 11 pairs. Every width-3 network involved is irreducible, so by Sussmann's theorem this search covers every exact symmetry.
3. **One spare neuron connects all symmetry copies** (proof; Section 5). Adding one hidden unit with zero output weight, a network is joined to each of its $2^r r!$ permutation and sign-flip copies by a piecewise-linear path of at most $10r - 6$ segments along which the network function is exactly constant for every input. The sign-flip move is, to my knowledge, new.

The hypotheses and main questions were fixed before any network was trained, every decision taken after seeing results is labelled as such, and one command, including tests, rebuilds every result except the optional exhaustive search in about 87 minutes on a laptop CPU, reproducing the main result files byte for byte (Section 9).

2 Background and related work

Barriers. For networks with parameters θ_A, θ_B and a loss or error L, the barrier along the straight line is [1, Eq. 1]

$$B(\theta_A, \theta_B) = \sup_{\alpha \in [0,1]} \Big[L\big(\alpha\theta_A + (1-\alpha)\theta_B\big) - \big(\alpha L(\theta_A) + (1-\alpha) L(\theta_B)\big) \Big]. \tag{1}$$

Entezari et al. report it at $\alpha = 1/2$ as a surrogate for the supremum [1, footnote 3]. I evaluate it on a grid of 25 values of α and use the training *error* as L for all verdicts (Section 3).

Symmetries of one-hidden-layer sigmoid networks. Write the network as

$$f(x) = \sum_{i=1}^{h} s_i \, \sigma(w_i^\top x + b_i) + c, \qquad \sigma(z) = \frac{1}{1 + e^{-z}}, \tag{2}$$

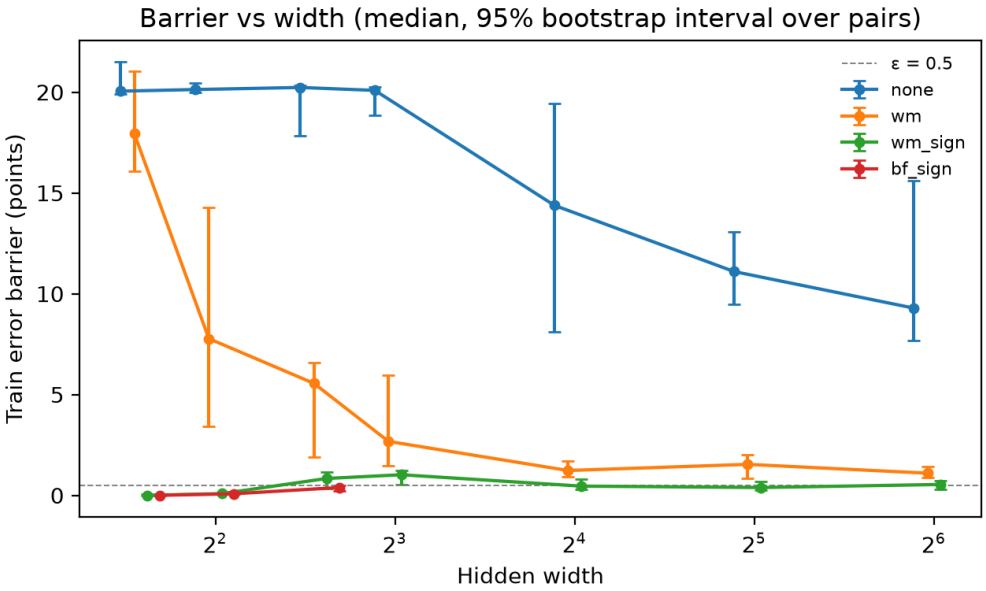


Figure 1: Training-error barrier against hidden width: median over independent pairs, with 95% bootstrap intervals (10,000 resamples of pairs). none: no alignment. wm: weight matching over permutations. wm_sign: weight matching over permutations and sign flips. bf_sign: brute force over all permutations and flips (widths 3–6), chosen by midpoint loss. Dashed line: $\varepsilon = 0.5$ points. Width 3: 11 pairs; widths 4–64: 20 pairs; no width-2 network met the inclusion rule.

with input weights w_i, biases b_i, output weights s_i and output bias c. Permuting the units leaves f unchanged [7]. Because $\sigma(-z) = 1 - \sigma(z)$, so does a *sign flip* of unit i:

$$(w_i, b_i, s_i, c) \mapsto (-w_i, -b_i, -s_i, c + s_i), \qquad \text{since} \qquad -s_i\sigma(-z) + c + s_i = s_i\sigma(z) + c. \tag{3}$$

Sussmann proved that for irreducible one-hidden-layer tanh networks these are the only symmetries [5]; Vlačić and Bölcskei restate his result as Lemma 2 of [6]. Since $\sigma(z) = \frac{1}{2}\big(1 + \tanh(z/2)\big)$, a sigmoid network with an output bias is a tanh network with rescaled weights, so the same holds for Eq. (2). A network is irreducible when no output weight is zero, no unit has zero input weights, and no two units have equal or opposite input parameters (w_i, b_i) [6, Lemma 1]; in practice I also flag units that are nearly constant on the data. Two limits matter here. First, uniqueness holds for the function on all of $\mathbb{R}^2$; on a finite data set, the set of equivalent networks can only be larger [6, Section I-A]. Bürgin et al. [8] study exactly such approximate, data-dependent symmetries and their effect on linear mode connectivity. Second, if a network is *nearly* reducible, for example because two units almost coincide, it has continuous near-symmetries that no finite search covers.

Related work. Entezari et al. ran a brute-force permutation search on small ReLU networks trained on MNIST [1, Appendix E.3], and Ainsworth et al. exhaustively searched the permutations of a hand-built two-hidden-layer ReLU network on a two-dimensional task to show that some solutions admit no linearly connecting permutation [3, Appendix A.6]. ReLU networks have no sign-flip symmetry, so flips do not arise in either setting. Boufalis et al. [9, Appendix A, Proposition 1] extended weight matching to sign flips for sine and tanh networks, essentially the wm_sign mode used here. They evaluated it on networks constructed to be equivalent (transformed copies of one network with small perturbations), and helpfully note that it can sometimes flip signs unnecessarily; their experiments on independently trained CNNs, whose focus was a different method, used permutation-only matching. Building on their work, I apply the sign-aware method to

independently trained networks and compare it with an exhaustive search. Şimşek et al. [10] showed that at minimal width the global minima are isolated permutation copies, and that one extra neuron connects them through straight segments (their Theorem 3.1 and Corollary 4.1); for sigmoid, they remark that "the analysis must include additional invariances" (Remark 4.1). Section 5 builds on their construction by adding a sign-flip move. Zhan et al. [11] analyze a two-layer ReLU teacher–student model with fixed output weights, no biases and a population loss; from their characterization of the global minima (Lemma 2, from [12]) they note that when student and teacher widths are equal, any two minima coincide up to permutation, so the permuted barrier is zero. My width-3 result can be seen as an empirical sigmoid analogue, in which the coincidence holds only up to permutations *and* sign flips. Zhao et al. [13] relate the connected components of the set of minima to the topology of the symmetry group, build constant-loss curves from continuous symmetries (their Section 6.1), and note that characterizing the minima of nonlinear networks "requires identifying the complete set of symmetries" (Section 7). For irreducible one-hidden-layer sigmoid networks that complete set of exact symmetries is known, and this study searches all of it. Ito et al. [14] show that very wide image networks become linearly connected without any permutation after temperature calibration; I return to their work in Section 6.

3 Setup and pre-registration

Task and data. The task is the "coffee roasting" problem from the Machine Learning Specialization [15]: two features (temperature 150–285 °C, duration 11.5–15.5 min), and a roast is good inside a triangle bounded by $t > 175$, $d > 12$ and $d \leq -\frac{3}{85}t + 21$. The data generator is a re-implementation written for this study; it uses the same labeling rule as the course's `load_coffee_data`, checked against a publicly available, unofficial copy of the lab file, since the official copy was not accessible to me; no course files are redistributed with this work. Unlike the lab, which uses 200 points, I draw 2,000 fresh training points and 2,000 test points (seeds 0 and 1); 20.95% of the training points are good roasts. Features are standardized with the training statistics. The good region has exactly three edges, so three hidden units are the natural minimum.

Networks and training. Networks follow Eq. (2) with a sigmoid output and binary cross-entropy. I trained widths 2, 3, 4, 6, 8, 16, 32 and 64, with 40 seeds each, using Glorot-uniform weights and zero biases [16] and full-batch Adam [17] (learning rate 0.01, 3,000 steps, no weight decay). Because training is full batch, the initialization is the only source of randomness. Before the main run I checked the learning rate against a rule fixed in advance (loss always finite; no rebound of more than 10% after step 200; final loss within 5% of the lowest); all 10 check runs (5 seeds at widths 3 and 64) passed.

Inclusion and pairing. A network is included if its training error is at most 2%. Included networks are paired disjointly in seed order, so pairs are independent samples. Inclusion was 0 of 40 at width 2, 23 of 40 at width 3 (11 pairs) and 40 of 40 at every larger width (20 pairs).

Barrier. I evaluate Eq. (1) at 25 evenly spaced values of α. The *training-error barrier*, in percentage points (1 point = 20 of 2,000 points), decides every verdict, with $\varepsilon = 0.5$ points (10 points of the training set) as the threshold for "removed". The test-error barrier is reported alongside. Loss barriers are reported only after output-scale calibration (Section 6).

Alignment modes. Before interpolating, network B is aligned to network A in one of five ways:

- `none`: no alignment.

Table 1: Pre-registered hypotheses (H) and questions (Q). A line "matches" if its angle differs by at most 10° and its offset by at most 0.25 standardized units.

	Prediction or question	Falsified if
H1	More than half of converged width-3 pairs learn the same three lines	Half or fewer of the pairs match on every line
H1b	Seeds with higher training error have lines further from the other seeds'	Spearman $\rho \leq 0$ over all 40 width-3 seeds
H2	The wm-aligned blend at width 3 is slightly worse, about 1.8 points	The median falls outside 0.5–3 points
H3	The aligned (wm) barrier shrinks with width	Width 64's median is not below width 3's (a rise-then-fall counts as a partial miss)
Q1	Do independently trained width-3 networks learn the same lines?	—
Q2	How does the barrier change with width?	—
Q3	Does weight matching find the barrier-minimizing alignment?	—
Q4	Do sign flips lower the barrier beyond permutations?	—

- wm: weight matching [3]. For one hidden layer this is a single linear assignment problem: I represent each unit by $u_i = (w_i, b_i, s_i) \in \mathbb{R}^4$ and choose the permutation maximizing $\sum_i u_i^{A\top} u_{\pi(i)}^B$ (Hungarian algorithm [18]).
- wm_sign: the same with sign flips allowed, solved by maximizing $\sum_i |u_i^{A\top} u_{\pi(i)}^B|$ and flipping unit $\pi(i)$ by Eq. (3) when its inner product is negative, as in [9, Proposition 1]. The output-bias shift of a flip is not part of the matching objective.
- bf_perm and bf_sign: brute force over all permutations (widths up to 8) or all permutations and flips (widths up to 6), choosing the alignment with the lowest loss at the midpoint, the surrogate of [1].

Two further modes, exact_perm and exact_sign, choose by the full 25-point error barrier instead; these belong to the post-hoc analysis of Section 4.3.

Pre-registration. The hypotheses and questions in Table 1 were fixed in my design sessions of 4–5 October 2026 and recorded in my research notebook before any network was trained. The numerical thresholds (inclusion rule, line-matching tolerances, ε, the H2 band, and the H1b distance and test) were dated 7–8 October, before the main run on 9 October. Three things were *not* dated in advance: the pass fractions named in the falsifiers of Q1, Q3 and Q4, and which aligned mode Q2 should use. None of the answers below depends on them except Q3, which I therefore report as counts only. This was an informal pre-registration: the record is a dated private research notebook, not a public registry such as OSF. The project was not under version control at the time, so there is no pre-registration commit; the dated notebook is the record, and it is available from me on request. Everything decided after seeing results is labelled *post-hoc* below.

4 Results

All values are medians over pairs with 95% percentile bootstrap intervals [19] (10,000 resamples of pairs); paired differences are medians of per-pair differences. Appendix A lists every value.

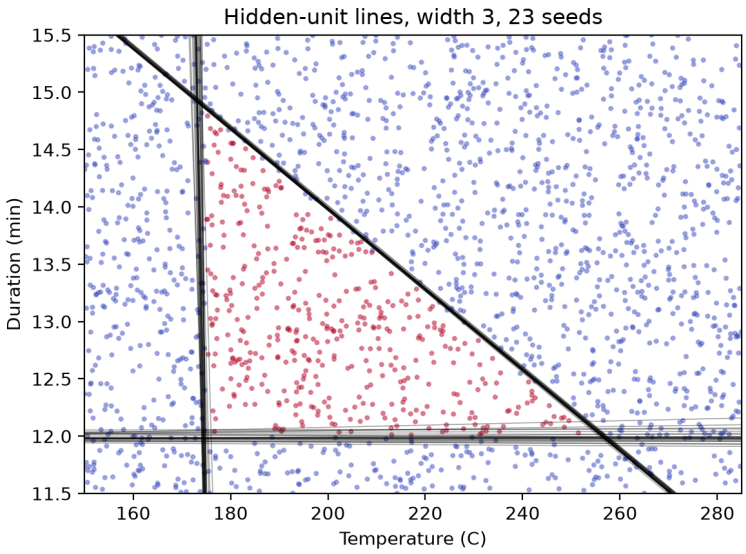


Figure 2: The three hidden-unit lines of all 23 included width-3 networks, drawn in original units over the training data (red: good roasts). Independently trained networks learn the same three edges of the triangle.

4.1 Width-3 networks learn the same three lines (H1 supported, H1b supported)

All 11 of 11 width-3 pairs matched on every line (largest angle difference 2.0°, largest offset 0.12; Figure 2), so H1 is supported and the answer to Q1 is yes. For comparison, 16 of 20 width-4 pairs matched. The claim covers networks that finished training: at a looser 5% inclusion rule, 10 of 12 width-3 pairs matched, and the two that did not match differed by 39.5° and 23.9°.

For H1b, the Spearman rank correlation [20] between a seed's training error and its median distance to the other seeds' lines is $\rho = 0.800$ [0.625, 0.886] over all 40 width-3 seeds, so H1b is supported. The support comes from two separate groups rather than a smooth trend: 23 seeds that finished training (error 0.65–1.3%, distance 0.20–0.52 threshold units) and 17 that did not (error 3.65–16.85%, distance 2.4–10.8). Within each group the correlation is weak ($\rho = 0.17$ and 0.42).

4.2 Permutations leave the barrier; sign flips remove it (H2 not supported, Q4 yes)

Although the width-3 networks compute the same three lines, the median wm barrier is **17.95 [16.07, 21.05] points**, far outside the 0.5–3 band I predicted, so H2 is not supported. Without alignment the median is 20.05 points. Reordering hidden units alone does not align these networks.

Allowing sign flips changes the picture (Figures 1 and 3). The pre-registered Q4 measure compares the best permutation (bf_perm) with the best permutation and flip pattern (bf_sign): flips lowered the training-error barrier by more than ε in **10 of 11** pairs at width 3, 16 of 20 at width 4 and 16 of 20 at width 6, with median drops of 17.79, 7.72 and 3.37 points (paired interval at width 3: -17.79 [-20.18, -15.14]). The best sign-aware alignment used at least one flip in 10 of 11 width-3 pairs. The wm_sign barrier at width 3 is 0.00 [0.00, 0.05]. Beyond the Q4 measure, the per-pair difference wm_sign $-$ wm has its whole interval below zero at every width, from -17.84 [-21.05, -16.07] at width 3 to -0.61 [-0.93, -0.38] at width 64.

The mechanism is visible in Eq. (3). Two networks that differ by a flip of unit i compute the same function, but halfway along the straight line between them that unit's input weights, bias and output weight all blend to zero. Its contribution to the logit then changes by $-s_i(\sigma(z) - \frac{1}{2})$, which is up to $|s_i|/2$ in size. On one trained width-3 network (seed 1, unit 2, $s = -11.14$), the straight blend to its own flipped copy changes

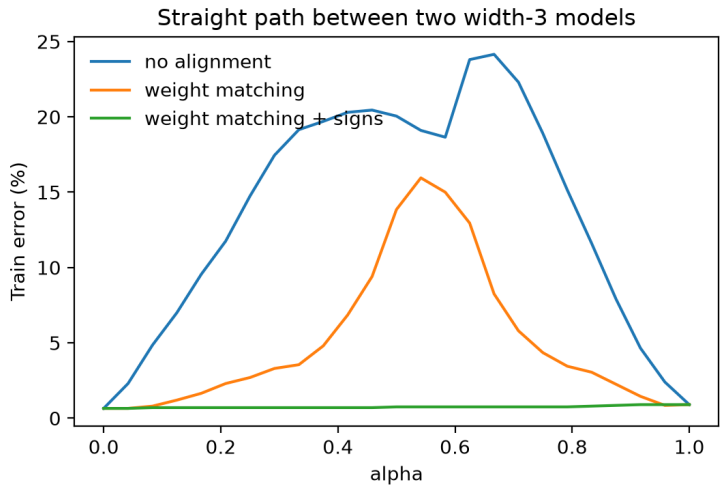


Figure 3: Training error along the straight path between one pair of width-3 networks, with no alignment, with weight matching, and with weight matching plus sign flips.

Table 2: Post-hoc complete-group search (exact_sign): training-error barrier left after the best of all $2^h h!$ permutation-and-flip combinations, chosen by the full barrier.

Width	Pairs	Residual: median / upper quartile / max	Pairs above ε	Pairs with a nearly reducible network
3	11	0.00 / 0.04 / 0.11	0	0 of 11
4	20	0.08 / 0.11 / 1.44	1	20 of 20
6	20	0.33 / 0.52 / 1.78	5	20 of 20

the logit by up to 5.57 and reaches 20.95% training error (Figure 4). Permutation matching cannot undo such a flip; it can only choose which unit is paired with which.

4.3 Post-hoc: an exhaustive search over the complete group

After the pre-registered questions were answered, I asked how large a barrier remains after searching *every* permutation and sign flip and choosing by the full 25-point error barrier rather than the midpoint. The rules for this analysis, including the irreducibility checks and the threshold for "removed" (upper quartile at most ε), were dated on 9 October, before the search ran. A network was flagged *nearly reducible* if its smallest output weight was below 5% of its largest, if two of its own units matched under the same line tolerance used for H1, or if a unit's activation varied by less than 0.01 (standard deviation) over the training data.

At width 3, the residual is at most **0.11 points** in all 11 pairs (Table 2). Every included width-3 network is irreducible: the closest pair of units within a network is at a median of 4.2 threshold units, where 1 would mean the same line. The search therefore covers every exact symmetry these networks have, and exact symmetries account for essentially the whole barrier. The result holds at 1% and 5% inclusion (maximum residual 0.09 and 0.47 points; no included network nearly reducible).

At widths 4, 6 and 8, nearly every network has two units on almost the same line, with a spare unit copying one of the three lines the task needs (37 of 40 networks at width 4, and all networks at widths 6 and 8). Such networks have continuous near-symmetries outside the finite group, so their residuals are upper bounds only; I stated this risk before the search. Width 6 falls short of the "removed" threshold by 0.02 (upper quartile 0.517). I decided after seeing the width-6 result not to run the signed search at width 8 (10.3 million candidates per pair), since its networks are all nearly reducible.

Table 3: Q3: weight matching against brute force (pairs where weight matching is the same / worse / better, within ε). Left: pre-registered comparison with midpoint-chosen brute force. Right: post-hoc comparison with the exact search. —: not computed; the signed search was not run at width 8 (10.3 million candidates per pair; see Section 4.3).

	Pre-registered (midpoint)		Post-hoc (exact)	
Width	wm vs. bf_perm	wm_sign vs. bf_sign	wm vs. exact_perm	wm_sign vs. exact_sign
3	8 / 2 / 1	11 / 0 / 0	7 / 4 / 0	11 / 0 / 0
4	11 / 4 / 5	19 / 1 / 0	15 / 5 / 0	19 / 1 / 0
6	14 / 6 / 0	11 / 9 / 0	12 / 8 / 0	11 / 9 / 0
8	9 / 10 / 1	—	8 / 12 / 0	—

For the question raised by Ainsworth et al. [3, Section 5.3], the answer *for these sigmoid networks on this task* is "another symmetry": permutation matching leaves a large barrier because a sign flip is not a permutation, and once flips are included, both weight matching and the exhaustive search remove the barrier at width 3. ReLU networks have no sign-flip symmetry, so this explanation does not carry over to the ReLU networks of their experiments, where the question remains open.

4.4 Width dependence (H3 supported, Q2)

The median wm barrier falls from 17.95 points at width 3 to **1.10 [0.86, 1.42]** at width 64, so H3 is supported. It rises by 0.31 points between width 16 (1.23) and width 32 (1.54). I judged after the run that this is not the falsifier's "rise-then-fall", because the curve does not first rise from width 3 and 0.31 is below ε; the intervals ([0.91, 1.68] and [0.83, 2.03]) overlap.

For Q2: without alignment the median barrier stays near 20 points up to width 8 and then falls to 9.29 [7.67, 15.60] at width 64. With wm it shrinks, but its upper quartile stays above ε at every width (1.61–20.31). With wm_sign the barrier is removed at widths 3 and 4 (upper quartiles 0.04 and 0.11), but not from width 6 to 64 (upper quartiles 0.77–1.30, medians 0.39–1.02). Reporting both aligned modes was a post-hoc decision, because Q2 did not name one. The test-set barrier gives the same answers at every width.

4.5 Does weight matching find the best alignment? (Q3, counts only)

Table 3 compares weight matching with brute force, counting a pair as "same" when the two barriers differ by at most ε. Under the pre-registered comparison, weight matching "beat" brute force in 7 pairs, by up to 20 points. This is possible only because brute force chose by the midpoint. Entezari et al. found the midpoint an accurate surrogate in their settings [1, footnote 3], but in these small networks the error peak sometimes lies elsewhere: in all 7 pairs it lay more than 0.05 from $\alpha = 0.5$ ($\alpha = 0.375$ to 0.708). The post-hoc exact search removes this artifact. With sign flips, weight matching finds the best alignment at widths 3 and 4 and drifts from it as width grows. With flips, the midpoint and full-barrier rules never differed by more than ε.

5 One spare neuron connects all symmetry copies

Two networks that differ by a sign flip compute the same function, yet, as shown above, the straight line between them has a large bump. This section shows that with one extra hidden unit there is a path between them along which the function never changes, and that combining this move with permutations reaches every symmetry copy.

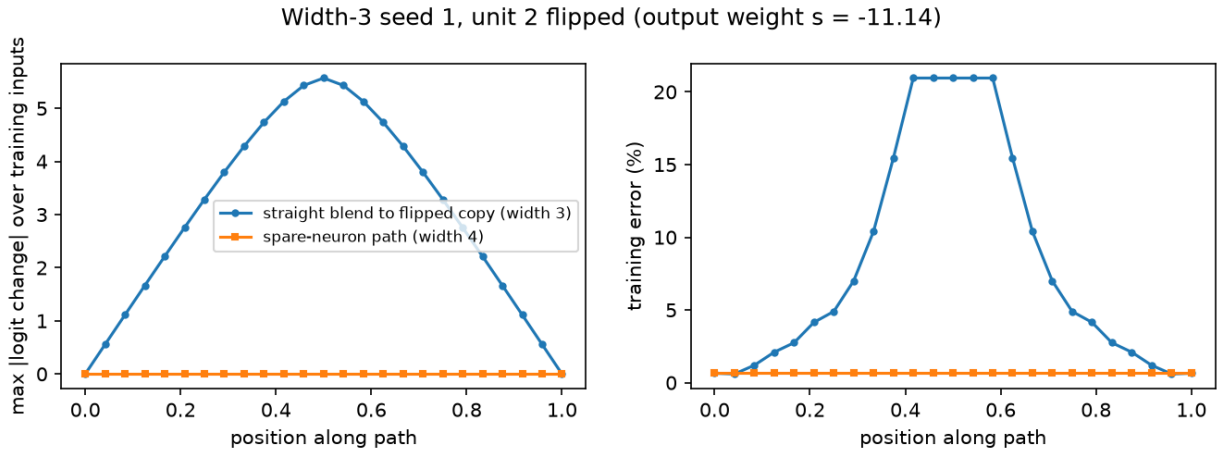


Figure 4: Width-3 network (seed 1), unit 2 flipped ($s = -11.14$). Left: largest change in the logit over the training inputs. Right: training error. Blue: straight blend to the flipped copy at width 3. Orange: the spare-neuron path of Lemma 1 at width 4, along which the function is unchanged.

Lemma 1 (Flip through a spare unit)**.** *Take unit i of Eq. (2) with parameters (w_i, b_i, s_i) and add a spare unit k with input parameters $(-w_i, -b_i)$ and output weight 0. For every t and every input, with $z = w_i^\top x + b_i$,*

$$(s_i - t)\,\sigma(z) \;-\; t\,\sigma(-z) \;+\; (c + t) \;=\; s_i\,\sigma(z) + c. \tag{4}$$

Proof. $-t\,\sigma(-z) + t = -t\,(1 - \sigma(z)) + t = t\,\sigma(z)$. □

Sliding t from 0 to s_i along this straight segment moves the whole contribution of unit i onto the spare: at $t = s_i$, unit i has output weight zero and the spare is exactly the sign-flipped unit i of Eq. (3). On the trained width-3 network of Section 4.2, this path changes the logit by at most 5×10^{-15} (rounding error) and keeps the training error at 0.65% throughout, while the straight blend to the flipped copy peaks at 20.95% (Figure 4).

Proposition 1 (One spare neuron connects all exact symmetry copies)**.** *Let f be a one-hidden-layer sigmoid network of width r with an output bias, embedded in width $r+1$ by adding a unit with output weight 0. Then each of the $2^r r!$ permutation and sign-flip copies of f, embedded in the same way (with any input parameters for the spare unit, which do not affect the function; one further segment sets them to a chosen value), is joined to f by a piecewise-linear path in parameter space with at most $10r - 6$ segments, along which the network function is exactly constant for every input.*

Proof. Three moves keep the function exactly constant along a straight segment:

(M1) A unit with output weight 0 may move its input weights and bias anywhere.

(M2) Two units with identical input weights and biases may trade output weight, keeping the sum fixed, since $(a + b)\,\sigma(z)$ depends only on $a + b$.

(M3) If unit k has input parameters $(-w_i, -b_i)$ and output weight 0, then Lemma 1 moves the contribution of unit i onto k, flipped.

Let k be the spare unit. A *transposition* of units i and j takes six segments: M1 (copy i's input parameters into k), M2 ($i \to k$), M1 (copy j's into i), M2 ($j \to i$), M1 (copy k's into j), M2 ($k \to j$); the spare again has output weight zero at the end. A *flip* of unit i takes four: M1 (set k to $(-w_i, -b_i)$), M3, M1 (copy k's input parameters into i), M2 ($k \to i$). Any permutation of r elements is a product of at most $r - 1$ transpositions, and any sign

pattern is a product of at most r single-unit flips, so every copy is reached in at most $6(r-1)+4r = 10r-6$ segments. □

The permutation part follows the idea of Şimşek et al. [10], whose construction connects permutation copies through straight segments; the flip move M3 is, to my knowledge, new. The construction was also checked numerically on a random width-3 network with 5,000 random inputs, applying two transpositions and two flips: the largest change in the output along the whole path was 5.3×10^{-15}, and the end point equalled the target copy exactly. Two limits apply. The proposition concerns a network embedded with an inactive spare unit (output weight zero), not two independently trained networks of width $r+1$. It shows that a flat path *exists*; it does not address the straight-line barrier, which is what the experiments measure. In the language of Zhao et al. [13], sign-flipped copies are separate components of the set of equivalent networks at width r, and one extra unit joins them with an explicit curve of constant function, in the spirit of their symmetry-induced curves (Section 6.1).

6 Pitfalls and a side-check

Training length inflates loss barriers. The coffee labels are separable, so cross-entropy keeps growing the weights: after 3,000 steps the median weight norm (input and output weights of included networks) rises from 32.2 at width 3 to 66.5 at width 64 (Appendix B). A misclassified point's loss grows with the weight scale, so raw loss barriers depend on training length and would mix that into the width trend. Zhao et al. make a related point precise: with a rescaling symmetry, two parameter settings of the same function can have an arbitrarily large loss barrier between them [13, Proposition 5.3]. I therefore base every verdict on the error barrier and report loss barriers only after choosing the best of 41 output scales (0.25 to 8, log-spaced) at each blend point, an idea similar in spirit to the temperature calibration of Ito et al. [14]. This range proved too narrow for 43 of 131 unaligned pairs, whose best scale sat at the lower edge, so calibrated losses for unaligned pairs are only partly calibrated. Aligned pairs almost never hit the edge.

Weight decay. Ito et al. [14] report that sufficiently wide image networks (MLPs, VGG-11 and ResNet-20 widened up to 16–32×) become linearly connected without permutations after temperature calibration, using weight decay 0.003 in most runs. With weaker decay (10^{-4}), VGG-11 and ResNet-20 show large barriers (their Section 5.4), although their Appendix F.9 notes that for sufficiently wide models, smaller weight decay gave smaller barriers. Their explanation is stated for bias-free ReLU networks with low-rank weights. My main run used no weight decay. As a side-check with a rule dated before the run, I trained 40 width-64 networks with L2 regularization 0.003 added to the gradient inside Adam (not decoupled weight decay, so not a replication of any published setting). **None of the 40 networks met the inclusion rule**: all stopped at 6.15–6.80% training error, with weights about 7 times smaller (median norm 9.4 against 66.5). With no included pairs, the dated rule gives no verdict. An exploratory comparison on all 40 networks, ignoring inclusion and decided after seeing this result, shows no clear change in the unaligned barrier (median 9.29 without decay against 14.32 with it; interval of the difference $[-1.30, 6.68]$). These networks are underfit and roughly a hundred times narrower than those of Ito et al., so this side-check does not bear on their findings.

7 Robustness

- **Inclusion threshold.** Rerunning the analysis at 1% and 5% inclusion changes no verdict. Two qualifications: at 1%, the width-3 paired intervals for the flip effect reach zero at their upper end ($\mathsf{wm_sign} - \mathsf{wm}$: -17.70 $[-22.16, 0.00]$), because 3 of 9 pairs needed no flip at all, although the count-based Q4 answer stands; at 5%, H1 holds with 10 of 12 pairs.

- **Test set.** The test-error barrier tracks the training-error barrier at every width (for example, wm at width 3: 17.95 training, 18.95 test).
- **Reproduction.** A clean copy of the project, run from one script, reproduced the main result files byte for byte (Section 9).

8 Limitations

- One architecture (one hidden layer, sigmoid), one two-dimensional task, widths 2 to 64, and one optimizer setting (3,000 full-batch Adam steps).
- "Exact symmetry" means equality of the function on all inputs. A network that matches another only on the data, an approximate symmetry in the sense of [6, 8], is not in the search and could lower the barrier further.
- The complete-group result is clean only at width 3. From width 4 upward the networks are nearly reducible, and the finite search no longer covers all near-symmetries.
- No width-2 network reached the inclusion threshold, so the minimal-width regime rests on width 3 alone, with 11 pairs.
- This study does not test the conjecture of Entezari et al., which concerns wide networks, nor the findings of Ito et al.
- Several decisions were taken after seeing results; each is labelled in the text and dated in my research notebook.

9 Conclusion

In the minimal-width sigmoid networks studied here, the barrier that permutation matching leaves behind is not a failure of matching but a missing symmetry. Sign flips account for it almost entirely: at width 3, permutations leave 17.95 points of training-error barrier, an exhaustive search over the complete exact group leaves at most 0.11, and one spare neuron turns each sign flip into a path of constant function. Three questions follow. First, what does a teacher–student analysis in the style of Zhan et al. [11] predict for sigmoid or tanh networks, where sign flips must enter the matching? Second, how much of the small residual barrier at widths 4 and above comes from near-symmetries of nearly reducible networks or from symmetries that hold only on the data? Third, does the picture survive depth, other tasks, and a weaker weight decay chosen by a rule fixed in advance?

Acknowledgments

This study builds directly on the work of Entezari et al., Ainsworth et al. and Benzing et al., whose clearly stated open questions motivated it, and on the theoretical results of Sussmann, Şimşek et al., Zhan et al. and Zhao et al. I am grateful to all of these authors for making their work openly available. The coffee-roasting task comes from the Machine Learning Specialization by Andrew Ng, DeepLearning.AI and Stanford Online.

Reproducibility statement

Code, result files and figures are released with this paper at https://github.com/MrigeshLakhanpal/sign-flips-lmc (archived on Zenodo; DOI to be added); the dated research notebook is available on

request. A single script (`run.sh` on Linux and macOS, `run.ps1` on Windows) runs the unit tests, the learning-rate check, training, every pre-registered and post-hoc analysis except the optional exhaustive search (`src/run_b1.py`, several hours), the bootstrap intervals, the figures, the inclusion sensitivity check, the spare-neuron figure and the weight-decay side-check. It took 87 minutes on a Windows laptop (CPU only, NumPy). Run from a clean copy of the project, it produced byte-identical `barriers.csv` and `h1b.csv` and identical report tables. Software: Python 3.12.10, NumPy 2.5.3, SciPy 1.18.1, pandas 3.0.5, Matplotlib 3.11.2. All randomness is seeded.

Authorship and use of AI assistance

The research questions and hypotheses are mine; I chose the pre-registered rules, several from options Claude proposed, and ran every experiment on my own machine. I used Claude (Anthropic) as an AI research and coding assistant: it wrote most of the code and analysis scripts, which I tested and parts of which I wrote or re-implemented myself as checks; it searched for and summarized related work, of which I read the sections this paper relies on; it wrote the proof of Proposition 1, whose lemma I derived on paper with its guidance and tested on my own trained network (Figure 4); and it drafted the text from my research notes, which I reviewed line by line and approved. A separate Claude instance audited every number against the result files. No third-party code is included. I am responsible for the content.

References

[1] Rahim Entezari, Hanie Sedghi, Olga Saukh, and Behnam Neyshabur. The role of permutation invariance in linear mode connectivity of neural networks. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2110.06296.

[2] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel M. Roy, and Michael Carbin. Linear mode connectivity and the lottery ticket hypothesis. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pages 3259–3269, 2020.

[3] Samuel K. Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git Re-Basin: Merging models modulo permutation symmetries. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2209.04836.

[4] Frederik Benzing, Simon Schug, Robert Meier, Johannes von Oswald, Yassir Akram, Nicolas Zucchet, Laurence Aitchison, and Angelika Steger. Random initialisations performing above chance and how to find them. In *14th Annual Workshop on Optimization for Machine Learning (OPT 2022)*, 2022. arXiv:2209.07509.

[5] Héctor J. Sussmann. Uniqueness of the weights for minimal feedforward nets with a given input-output map. *Neural Networks*, 5(4):589–593, 1992.

[6] Verner Vlačić and Helmut Bölcskei. Affine symmetries and neural network identifiability. arXiv:2006.11727, 2020.

[7] Robert Hecht-Nielsen. On the algebraic structure of feedforward network weight spaces. In *Advanced Neural Computers*, pages 129–135. Elsevier, 1990.

[8] Vincent Bürgin, Daniel Herbst, Ya-Wei Eileen Lin, and Stefanie Jegelka. Beyond structural symmetries: Linear mode connectivity via neuron identifiability. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *PMLR*, 2026. arXiv:2606.04754.

[9] Odysseas Boufalis, Jorge Carrasco-Pollo, Joshua Rosenthal, Eduardo Terrés-Caballero, and Alejandro García-Castellanos. Symmetry-aware graph metanetwork autoencoders: Model merging through parameter canonicalization. In *Proceedings of the 1st Conference on Topology, Algebra, and Geometry in Data Science (TAG-DS)*, 2025. arXiv:2511.12601.

[10] Berfin Şimşek, François Ged, Arthur Jacot, Francesco Spadaro, Clément Hongler, Wulfram Gerstner, and Johanni Brea. Geometry of the loss landscape in overparameterized neural networks: Symmetries and invariances. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *PMLR*, pages 9722–9732, 2021. arXiv:2105.12221.

[11] Keyao Zhan, Puheng Li, and Lei Wu. Analyzing the role of permutation invariance in linear mode connectivity. In *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 258 of *PMLR*, 2025. arXiv:2503.06001.

[12] Zhanran Lin, Puheng Li, and Lei Wu. Exploring neural network landscapes: Star-shaped and geodesic connectivity. arXiv:2404.06391, 2024.

[13] Bo Zhao, Nima Dehmamy, Robin Walters, and Rose Yu. Understanding mode connectivity via parameter space symmetry. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, 2025. arXiv:2505.23681.

[14] Akira Ito, Masanori Yamada, Daiki Chijiwa, and Atsutoshi Kumagai. Do we really need permutations? Impact of model width on linear mode connectivity. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.08023.

[15] Andrew Ng, DeepLearning.AI, and Stanford Online. Machine Learning Specialization, Course 2 (Advanced Learning Algorithms), Week 1 optional lab: coffee roasting. Coursera, 2022.

[16] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 9 of *PMLR*, pages 249–256, 2010.

[17] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[18] Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.

[19] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

[20] Charles Spearman. The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1):72–101, 1904.

A Full results

Table 4: Training-error barrier (points; median [95% bootstrap interval over pairs]), calibrated loss barrier (median) and test-error barrier (median, points) for every width and alignment mode. Generated directly from the result files.

Width	Mode	Pairs	Training-error barrier	Cal. loss barrier	Test-error barrier
3	bf_perm	11	17.88 [15.16, 20.22]	0.355	18.90
	bf_sign	11	0.00 [0.00, 0.05]	0.000	0.02
	none	11	20.05 [19.90, 21.50]	0.393	20.84
	wm	11	17.95 [16.07, 21.05]	0.283	18.95
	wm_sign	11	0.00 [0.00, 0.05]	0.000	0.02
4	bf_perm	20	7.77 [3.43, 15.39]	0.199	8.69
	bf_sign	20	0.08 [0.04, 0.10]	0.000	0.06
	none	20	20.13 [19.97, 20.44]	0.392	20.86
	wm	20	7.77 [3.43, 14.30]	0.227	8.69
	wm_sign	20	0.09 [0.04, 0.11]	0.000	0.06
6	bf_perm	20	4.68 [1.71, 6.15]	0.082	5.20
	bf_sign	20	0.39 [0.21, 0.48]	0.005	0.55
	none	20	20.23 [17.83, 20.33]	0.335	20.92
	wm	20	5.55 [1.89, 6.57]	0.106	5.98
	wm_sign	20	0.84 [0.34, 1.17]	0.012	1.08
8	bf_perm	20	1.94 [1.30, 2.98]	0.039	2.44
	none	20	20.09 [18.85, 20.24]	0.352	20.64
	wm	20	2.68 [1.46, 5.97]	0.070	3.55
	wm_sign	20	1.02 [0.54, 1.24]	0.017	1.05
16	none	20	14.38 [8.12, 19.45]	0.264	15.44
	wm	20	1.23 [0.91, 1.68]	0.025	1.45
	wm_sign	20	0.46 [0.31, 0.79]	0.008	0.57
32	none	20	11.10 [9.49, 13.08]	0.217	11.95
	wm	20	1.54 [0.83, 2.03]	0.033	1.61
	wm_sign	20	0.39 [0.26, 0.68]	0.007	0.28
64	none	20	9.29 [7.67, 15.60]	0.197	10.42
	wm	20	1.10 [0.86, 1.42]	0.023	1.20
	wm_sign	20	0.54 [0.31, 0.72]	0.010	0.45

Table 5: Weight matching minus brute force, per pair (points; median [95% interval]). Positive: weight matching is worse.

Width	Comparison	Pairs	Median per-pair difference
3	wm − bf_perm	11	0.00 [0.00, 0.00]
4	wm − bf_perm	20	0.00 [0.00, 0.00]
6	wm − bf_perm	20	0.01 [0.00, 0.77]
8	wm − bf_perm	20	0.63 [0.08, 1.34]
3	wm_sign − bf_sign	11	0.00 [0.00, 0.00]
4	wm_sign − bf_sign	20	0.00 [0.00, 0.00]
6	wm_sign − bf_sign	20	0.26 [0.00, 0.62]
3	wm − exact_perm	11	0.00 [0.00, 0.90]
4	wm − exact_perm	20	0.00 [0.00, 0.00]
6	wm − exact_perm	20	0.42 [0.05, 1.38]
8	wm − exact_perm	20	1.03 [0.42, 2.42]
3	wm_sign − exact_sign	11	0.00 [0.00, 0.00]
4	wm_sign − exact_sign	20	0.00 [0.00, 0.00]
6	wm_sign − exact_sign	20	0.34 [0.00, 0.66]

Table 6: With sign flips minus without, per pair (points; median [95% interval]). Negative: flips lower the barrier.

Width	Comparison	Pairs	Median per-pair difference
3	wm_sign − wm	11	−17.84 [−21.05, −16.07]
4	wm_sign − wm	20	−7.72 [−14.26, −3.30]
6	wm_sign − wm	20	−3.67 [−5.81, −0.91]
8	wm_sign − wm	20	−1.32 [−4.55, −0.54]
16	wm_sign − wm	20	−0.76 [−1.05, −0.36]
32	wm_sign − wm	20	−1.00 [−1.36, −0.47]
64	wm_sign − wm	20	−0.61 [−0.93, −0.38]
3	bf_sign − bf_perm	11	−17.79 [−20.18, −15.14]
4	bf_sign − bf_perm	20	−7.72 [−15.31, −3.35]
6	bf_sign − bf_perm	20	−3.37 [−5.80, −1.15]
3	exact_sign − exact_perm	11	−17.79 [−20.02, −15.14]
4	exact_sign − exact_perm	20	−7.72 [−12.26, −3.35]
6	exact_sign − exact_perm	20	−2.83 [−5.58, −0.93]

B Weight norms and the weight-decay side-check

Table 7: Median weight norm (input and output weights, biases excluded) of included networks after 3,000 steps, no weight decay.

Width	3	4	6	8	16	32	64
Median norm	32.19	33.69	38.56	41.02	46.88	54.96	66.50

Table 8: Exploratory weight-decay comparison at width 64 (all 40 networks, inclusion ignored; 20 pairs with the same initializations as the main run). Medians; interval for median(decay) − median(base). Decided after seeing that no decay network met the inclusion rule.

Mode	Measure	No decay	L2 0.003	95% interval
none	error barrier (points)	9.29	14.32	[−1.30, 6.68]
none	calibrated loss barrier	0.197	0.244	[−0.024, 0.074]
wm	error barrier (points)	1.10	1.31	[−0.67, 2.20]
wm	calibrated loss barrier	0.023	0.020	[−0.013, 0.016]
wm_sign	error barrier (points)	0.54	0.12	[−0.68, 0.32]
wm_sign	calibrated loss barrier	0.010	0.002	[−0.015, 0.001]

C Timeline of decisions

Table 9: When each decision was taken. Post-hoc decisions are marked.

Date (2026)	Decision
4–5 Oct	Hypotheses H1–H3 and questions Q1–Q4 fixed and recorded, before any training
7 Oct	Learning-rate rule written, then checked (10 of 10 runs passed); coffee rule compared with an unofficial public copy of the lab file
8 Oct	Inclusion rule (2%), line tolerances (10°, 0.25), $\varepsilon = 0.5$, H2 band (0.5–3), H1b distance and test; training-error barrier chosen as the deciding quantity
9 Oct	Main run; pre-registered verdicts written; inclusion sensitivity (1% and 5%); rules for the complete-group search dated before it ran
9–10 Oct	Complete-group search; decision not to run the signed width-8 search (post-hoc)
10 Oct	Weight-decay rule dated
11 Oct	Weight-decay run; exploratory comparison decided after the result (post-hoc); bootstrap intervals; independent numeric audit of the draft; reproduction from a clean copy; proof of Proposition 1